Monotonicity as an Architectural Bias for Robust Language Models

Anonymous authors
Paper under double-blind review

Abstract

Large language models (LLMs) are known to exhibit brittle behavior under adversarial prompts and jailbreak attacks, even after extensive alignment and fine-tuning. This fragility reflects a broader challenge of modern neural language models: small, carefully structured perturbations in high-dimensional input spaces can induce large and unpredictable changes in internal semantic representations and output.

We investigate monotonicity as an architectural inductive bias for improving the robustness of Transformer-based language models. Monotonicity constrains semantic transformations so that strengthening information, evidence, or constraints cannot lead to regressions in the corresponding internal representations. Such order-preserving behavior has long been exploited in control and safety-critical systems to simplify reasoning and improve robustness, but has traditionally been viewed as incompatible with the expressivity required by neural language models.

We show that this trade-off is not inherent. By enforcing monotonicity selectively in the feed-forward sublayers of sequence-to-sequence Transformers—while leaving attention mechanisms unconstrained—we obtain monotone language models that preserve the performance of their pretrained counterparts. This architectural separation allows negation, contradiction, and contextual interactions to be introduced explicitly through attention, while ensuring that subsequent semantic refinement is order-preserving. Empirically, monotonicity substantially improves robustness: adversarial attack success rates drop from approximately 69% to 19%, while standard summarization performance degrades only marginally. We further show that the robustness benefit transfers to a decoder-only foundation model (Pythia-1.4B), but only when the constraint scope is matched to the architecture: constraining the feed-forward input projection alone—sufficient in the encoder-decoder setting—fails outright, whereas constraining both feed-forward projections reduces the mean attack success rate from 69.0% to 42.5% across three seeds, at a larger cost in language-modeling perplexity. Monotonicity is thus a practical inductive bias for robustness, with an architecture-dependent quality–robustness trade-off.

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language tasks, yet their behavior under adversarial or carefully structured inputs remains brittle. Even models that have undergone extensive alignment and fine-tuning can be induced to produce unsafe or unintended outputs through jailbreak prompts or small, targeted perturbations. This phenomenon is now well documented and points to a deeper underlying issue: modern language models operate in extremely high-dimensional spaces, where even subtle changes to the input or internal representations can induce large and difficult-to-predict changes in the output. Recent work has attributed this sensitivity, in part, to the high Lipschitz constants exhibited by such models, which amplify perturbations as they propagate through the network Newhouse et al. (2025).

This fragility has motivated a line of research on improving the robustness of language models Salah (2026); Su et al. (2024). Existing approaches largely focus on training-time interventions, such as alignment objectives Cao et al. (2024) or adversarial training Howe et al. (2024), as well as inference-time defenses such as output filtering Khachaturov & Mullins (2025). While these methods can be effective empirically, they are inherently reactive, addressing vulnerabilities after they are discovered rather than constraining model behavior by design. As language models are increasingly deployed, this raises a question: can we endow language models with structural properties that make their behavior more predictable under perturbation?

We investigate monotonicity as a structural inductive bias for improving the robustness of language models. At a high level, monotonicity constrains a model so that its outputs change in a predictable direction with respect to changes in its inputs. This property has a long history in machine learning Gupta et al. (2016); Runje & Shankaranarayana (2023) and related fields Smith (1995), and has been especially influential in control theory, where monotone systems admit strong guarantees on stability and safety even in very high-dimensional settings Michel et al. (2015); Ito et al. (2014). For example, monotonicity has been used to analyze and certify the behavior of large-scale dynamical systems such as power grids, where direct reasoning over all possible system states is infeasible Rantzer & Bernhardsson (2014).

Beyond its role in classical dynamical systems, monotonicity also aligns naturally with how we expect language models to behave at the level of semantic transformation. Many linguistic and reasoning operations are inherently order-preserving: adding supporting evidence should not weaken a conclusion; adding constraints or safety instructions should not result in less constrained internal representations; and increasing the stated severity or risk of a situation should not lead to a more permissive response. Yet contemporary language models frequently violate these expectations, exhibiting behavior in which the addition of seemingly clarifying or restrictive information leads to unpredictable or regressive changes in output.

Concrete examples of semantic monotonicity arise naturally in language understanding. Consider a sequence of prompts that progressively strengthen a factual claim ("The medication has side effects" $\preceq$ "The medication has severe side effects" $\preceq$ "The medication has severe side effects and requires medical supervision"), or that add explicit safety constraints ("Explain this process" $\preceq$ "Explain this process safely" $\preceq$ "Explain this process safely without providing actionable steps"). Semantically, such prompts form a partial order: later prompts contain strictly more information or stricter constraints. A model's internal representations should reflect this ordering, and its outputs should respect it. When models fail to do so, the cause is often not the absence of relevant information, but the way semantic features are transformed and recombined internally.

This perspective highlights a structural source of brittleness in modern architectures. Transformer models interleave contextual aggregation, via attention, with nonlinear feed-forward transformations that refine token-level representations. While attention mechanisms are well suited to introducing contextual operators such as negation, contrast, or exception by explicitly linking tokens and defining scope, the subsequent feed-forward transformations are unconstrained and may arbitrarily amplify, suppress, or cancel features. As a result, semantic reversals can occur implicitly through distributed feature cancellation rather than being driven by explicit linguistic signals. We argue that this implicit cancellation is a key contributor to brittle behavior under adversarial or carefully structured prompts.

Our motivation for monotonicity draws a close conceptual parallel to monotone Boolean circuits Jukna et al. (2012). In monotone circuits, logical negation is disallowed within the circuit itself; any negation must be introduced explicitly at the inputs. This restriction does not eliminate expressive power, but enforces a semantic discipline: truth cannot be undone by downstream computation. Analogously, enforcing monotonicity in semantic transformations encourages language models to represent negation, contradiction, or constraint relaxation explicitly through contextual mechanisms such as attention, rather than implicitly through fragile cancellations inside nonlinear transformations. Monotonicity thus serves as a design principle that separates where semantic reversals may occur from where meaning is refined, yielding predictable, auditable, and robust representations.

Viewed through this lens, a language model can be understood as a high-dimensional system whose internal semantic state evolves through successive transformations in response to input perturbations. Imposing monotonic structure on these transformations limits how adversarial variations propagate, ensuring that strengthening information or constraints cannot lead to regressions in meaning. As in monotone dynamical systems, this structure simplifies reasoning about global behavior. In such settings, it suffices to verify satisfaction only at extreme points Feyzmahdavian et al. (2017).

Our central finding is that monotonicity can be incorporated into Transformer architectures without sacrificing performance by enforcing it selectively within feed-forward sublayers, while leaving attention mechanisms unconstrained. This design preserves expressivity while imposing a disciplined form of semantic refinement. The resulting monotone Transformers match the task performance of their non-monotone counterparts and exhibit substantially improved robustness to adversarial and jailbreak attacks. In particular, these gains arise purely from architectural structure, without additional data, modified training objectives, or heuristic defenses, isolating monotonicity itself as a causal factor.

These results challenge the prevailing view, articulated in prior studies of constrained monotonic networks Sartor et al. (2025), that enforcing strong architectural constraints inevitably degrades optimization or expressive capacity in large neural models. Instead, they suggest that monotonicity can serve as a practical and principled inductive bias for shaping semantic behavior. Motivated by this perspective, we investigate how monotonicity can be imposed as a structural property of language models in a manner that is both semantically meaningful and practically viable. In particular, we focus on enforcing monotonicity at the level of semantic refinement, rather than as a global constraint, and develop this approach through both formal analysis and empirical evaluation.

Contributions. Our contributions are as follows:

1. Monotone Transformer components. We introduce monotone feed-forward sublayers for Transformers, enforcing structural monotonicity while leaving attention mechanisms unconstrained.
2. Lossless distillation. We show that pretrained Transformers can be distilled into monotone counterparts without degrading standard benchmark performance.
3. Improved robustness. We empirically demonstrate that monotone language models exhibit increased robustness to adversarial and jailbreak attacks.
4. Architecture-conditional transfer. We characterize how the robustness benefit of monotonicity transfers from encoder-decoder to decoder-only architectures, showing that decoder-only models require broader constraint coverage to close adversarial leak paths, and quantifying the resulting quality–robustness trade-off on Pythia-1.4B.

2 Related Work

Vulnerabilities of Language Models. Large language models are highly susceptible to adversarial manipulations that bypass alignment and safety mechanisms. Prior work has documented jailbreak prompts and transferable adversarial suffixes that generalize across models and settings, including aligned systems (Zou et al., 2023). Subsequent studies identify additional failure modes, such as position-sensitive jailbreaks (Wang et al., 2025b), interpretable attacks that evade perplexity-based defenses (Zhu et al., 2023), and many-shot jailbreaks whose effectiveness increases with larger context windows (Anil et al., 2024). Robustness benchmarks further show that models perform poorly under systematic adversarial evaluation (Wang et al., 2021), and that safety mechanisms often fail in multilingual or low-resource settings (Deng et al., 2024). Prompt injection attacks, which exploit the lack of separation between instruction and context, are now recognized as a fundamental vulnerability, alongside broader threats to inference and training-time surveyed in recent security analyses (Li & Fung, 2025).

Defenses and Robustness Enhancements. Proposed defenses include smoothing-based methods with provable jailbreak guarantees (Robey et al., 2023), lightweight detection based

on output distributions (Sayeedi et al., 2025), and adversarial training approaches whose effectiveness depends on model scale and threat model (Howe et al., 2024). Other work examines attack transferability and adaptive prompt sequences to harden models against structured adversarial dependencies (Wang et al., 2025a). Despite these advances, most defenses remain reactive and training-dependent.

Monotonic Neural Networks and Structural Constraints. Monotonicity has long been studied as a structural inductive bias for interpretability and robustness. Classical results show that feed-forward networks with non-negative weights and monotone activations are universal approximators of monotone functions, establishing that monotonicity need not limit expressivity (Sill, 1998; Daniels & Velikova, 2010). Subsequent work proposed practical monotone and partially monotone architectures, including lattice-based models, enabling scalable learning under monotonic constraints in structured domains (Gupta et al., 2016; You et al., 2017). More recent research highlights monotonicity's benefits for verification, reducing reasoning to boundary inputs in piecewise linear networks (Weber et al., 2021). More broadly, architectural constraints and structured layers have been shown to improve robustness and stability without prohibitive performance loss (Amos & Kolter, 2017). Our work builds on this line by showing that selectively enforced monotonicity can scale to Transformer-based language models and yield tangible robustness gains.

3 Monotone Transformers

We study the role of monotonicity constraints in improving the adversarial robustness of sequence-to-sequence (Seq2Seq) language models.

Definition 3.1 (Sequence-to-Sequence Model). A sequence-to-sequence model is a parameterized mapping

$$\mathcal{M}_\theta : \mathcal{X} \to \mathcal{Y},$$

where $\mathcal{X}$ denotes the space of input token sequences and $\mathcal{Y}$ denotes the space of output token sequences. In this work, $\mathcal{M}_\theta$ is instantiated as a Transformer architecture composed of attention layers, feed-forward network (FFN) sublayers, residual connections, and normalization operators.

We formalize monotonicity directly on hidden-state coordinates. For a hidden state $h \in \mathbb{R}^d$, equip $\mathbb{R}^d$ with the standard coordinatewise (product) order $h \le h'$, and interpret $h' \ge h$ as strengthening every hidden coordinate. This is the order with respect to which our FFN sublayers (defined below) are constructed to be monotone.

Section 4.3 additionally asks a distinct empirical question: whether this per-sublayer, coordinatewise property induces order preservation along interpretable semantic directions—for instance, whether strengthening a factual claim or adding a safety constraint (Section 1) yields hidden states that are ordered along directions $A \in \mathbb{R}^{p\times d}$ recovered post hoc via linear probes trained on ordered prompt pairs. That measurement is diagnostic only: the probe directions A are fit after training and play no role in defining or enforcing monotonicity below.

Definition 3.2 (Monotonicity). A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is monotone if for all $x, x' \in \mathbb{R}^n$,

$$x \le x' \quad \Rightarrow \quad f(x) \le f(x').$$

Lemma 3.3 (Closure under Composition). If $f : \mathbb{R}^n \to \mathbb{R}^m$ and $g : \mathbb{R}^m \to \mathbb{R}^k$ are monotone, then their composition $g \circ f$ is monotone.

Consider a feed-forward neural network $N : \mathbb{R}^{n_0} \to \mathbb{R}^{n_k}$ with k hidden layers. For input $u \in \mathbb{R}^{n_0}$,

$$\begin{aligned} y_0 &= u, \\ y_{i+1} &= \sigma(W_i y_i + b_i), \quad i = 0, \dots, k-1, \\ N(u) &= W_k y_k + b_k, \end{aligned}$$

where σ is applied elementwise.

Proposition 3.4 (Sufficient Condition for FFN Monotonicity). If σ is elementwise non-decreasing and all weight matrices satisfy

$$W_i \geq 0 \quad \text{for } i = 0, \ldots, k,$$

then N is monotone.

Proof. Each affine map with nonnegative weights is monotone, and σ preserves monotonicity by assumption. The claim follows by Lemma 3.3. □

Neural networks satisfying Proposition 3.4 are commonly referred to as monotone neural networks. Such networks are universal approximators of continuous monotone functions on compact domains Daniels & Velikova (2010).

Definition 3.5 (Monotone Transformer). A Transformer model $\mathcal{M}_\theta$ is monotone with respect to its feed-forward sublayers if each FFN sublayer implements a residual update

$$F(h) := h + N(h), \qquad N(h) = W_2\,\sigma(W_1 h + b_1) + b_2,$$

where W_1, W_2 satisfy the conditions of Proposition 3.4 (elementwise nonnegative, with σ elementwise non-decreasing), so that N is monotone on $\mathbb{R}^d$ with respect to the coordinatewise order. All other components (attention, residuals, normalization) are left unconstrained.

Because the identity map $h \mapsto h$ is trivially monotone and a sum of monotone vector-valued maps under a common order is monotone, $F = \mathrm{id} + N$ is itself monotone whenever N is; the residual connection therefore does not need a separate constraint. This definition does not assert global monotonicity of the full Transformer, but enforces monotonicity coordinatewise at the level of FFN sublayers, where implicit feature cancellation is most likely to occur Runje & Shankaranarayana (2023); Sartor et al. (2025); Nguyen et al. (2023).

We enforce the nonnegativity condition of Proposition 3.4 by constraining every FFN weight matrix (W_1 and W_2 above) to be elementwise nonnegative, using the (elementwise non-decreasing) activation already present in the pretrained network. Concretely, for each constrained weight matrix $W \in \mathbb{R}^{m \times n}$ we reparameterize

$$W = \phi(V), \qquad \phi(v) = \log(1 + \exp(v)),$$

with unconstrained V, applied elementwise. This guarantees $W \geq 0$ without projection or constrained optimization, and is maintained throughout training by construction rather than by post-hoc projection.

Naively initializing V so that $W \approx 0$ would discard the pretrained weights entirely. Instead we initialize V so that $\phi(V)$ matches the magnitude of the pretrained weight, $\phi(V_{\text{init}}) = |W_{\text{pre}}| + \epsilon$ for a small $\epsilon > 0$ (i.e., $V_{\text{init}} = \phi^{-1}(|W_{\text{pre}}| + \epsilon)$), which preserves the scale of the pretrained transformation while flipping the sign of every negative weight to satisfy $W \geq 0$. Biases are left at their pretrained values. Because roughly half of a typical pretrained weight matrix is negative, this initialization substantially changes the function computed by each FFN sublayer relative to the pretrained network—consistent with the elevated initial training loss reported in Table 1—even though it preserves more information than initializing from a small random matrix would.

4 Experimental Setup

We instantiate the monotone Transformer framework of Section 3 using the T5 architecture Raffel et al. (2020), enabling a controlled empirical study of monotonicity as an architectural inductive bias for robustness.

Implementation Scope. We apply Definition 3.5 to every Transformer FFN sublayer by reparameterizing both of its weight matrices (W_1, the input projection, and W_2, the output projection) with the elementwise-nonnegative softplus parameterization above, retaining the pretrained (non-decreasing) activation between them. Attention layers, residual connections, and normalization remain unconstrained.

We use T5-small, comprising 6 encoder and 6 decoder layers with 512-dimensional hidden representations, 8 attention heads per layer, and 2048-dimensional FFN intermediate activations. Of the model's approximately 60M parameters, roughly 24M (40%) correspond to

the FFN sublayers; every weight in these sublayers is constrained to be elementwise nonnegative, so monotonicity is enforced coordinatewise on the full 512-dimensional hidden state at each FFN sublayer, with no bottleneck or subspace restriction.

Training and evaluation protocols, including optimization hyperparameters, datasets, and decoding settings, are detailed in Appendix A.2.

4.1 Adversarial Robustness Evaluation

We evaluate robustness under two complementary classes of adversarial attacks designed to probe different failure modes of sequence-to-sequence models.

Universal Adversarial Triggers. Universal adversarial triggers (UATs) Wallace et al. (2019) consist of short, input-agnostic token sequences that, when prepended to an input, maximize the model's loss. We optimize triggers using coordinate-wise search with three random restarts and 50 iterations. Candidate tokens are drawn from a vocabulary biased toward punctuation, special characters, and high-frequency words that empirically induce greater disruption. Triggers are optimized on a held-out validation split and evaluated on a disjoint test set. To assess transferability, we additionally construct a transfer matrix measuring cross-model vulnerability to learned triggers.

HotFlip Attacks. HotFlip attacks Ebrahimi et al. (2018) identify vulnerable input positions by computing gradients of the loss with respect to token embeddings and replacing selected tokens to maximize loss increase. We allow up to five token replacements per example, selecting replacements based on the dot product between gradient directions and candidate embeddings.

Robustness Metrics. Robustness is quantified using ROUGE score degradation under attack, attack success rate (defined as a ROUGE-L degradation exceeding 10%), and statistical significance assessed via paired t-tests with Bonferroni correction and paired Cohen's d. We additionally report mean loss increase and analyze cross-model transferability of adversarial triggers.

4.2 HotFlip Substitution Transfer and Gradient-Free Controls

A success-rate gap between models whose attacks are each optimized against their own gradients could reflect a genuine difference in the functions they compute, or an artifact of each attack exploiting its source model's particular loss landscape. To separate these possibilities we persist the token substitutions found by HotFlip on each source model and replay those exact substitutions as fixed inputs against every target, yielding a 2×2 (crafted-on $\times$ evaluated-on) transfer matrix for the baseline and monotone models. The diagonal of this matrix reproduces the same-model HotFlip evaluation; the off-diagonal cells test whether substitutions found on one model remain damaging on the other.

Two gradient-free controls accompany the transfer matrix. A random-substitution control applies the same number of flips (five for T5, ten for Pythia) at uniformly random positions, with replacements drawn from the same restricted candidate vocabulary used by the UAT attack, and involves no model or gradient in generating the substitutions. A query-based control performs a greedy per-position search that scores candidate flips by the target model's loss directly, with no gradients, so that a robustness gap that survives this control cannot be attributed to gradient masking of the HotFlip optimizer. The same protocol is used on both the T5 and Pythia tracks.

4.3 End-to-End Order Preservation

Definition 3.5 enforces coordinatewise monotonicity at each FFN sublayer, not along semantically interpretable directions of the hidden state. To test whether that per-sublayer property induces order preservation end to end, we construct a templated corpus of ordered prompt pairs that progressively strengthen along thirteen semantic axes (severity, safety constraint, certainty, urgency, obligation, and related directions) instantiated over eighteen topics, yielding 234 chains. Adjacent levels of each chain form 702 ordered pairs (x, x'), of

which 195 (from a disjoint set of chains) are held out for evaluation so that probe directions are never fit on the pairs used to measure order.

For each model we extract encoder hidden states at every layer (embeddings plus each of the six T5 encoder layers), pooled by mean over token positions, with final-token pooling as a robustness check. On the fit split (507 pairs) we train $p = 64$ logistic-regression probe directions $A \in \mathbb{R}^{p \times d}$ on the final-layer representations; these probes are a post-hoc measurement instrument and play no role in training or in the monotonicity constraint. On the evaluation split we report, per layer, the fraction of the p probe coordinates satisfying $Ah \leq Ah'$ for each ordered pair, with bootstrap 95% confidence intervals. A per-layer fraction near one means the network's semantic coordinates increase along the expected direction at that depth; a fraction that degrades with depth would indicate that unconstrained attention, LayerNorm, and residual mixing lose order even though each individual FFN sublayer is coordinatewise monotone.

4.4 Attribution Ablations

The softplus reparameterization of Section 3 changes two things relative to the pretrained network: it imposes elementwise nonnegativity (the monotonicity constraint), and it disrupts the pretrained weight geometry by initializing from $|W_{\text{pre}}|+\epsilon$. Parameter count is unchanged, so a rank or parameter-count confound does not arise. Two control arms isolate these factors while holding the training protocol, optimizer, and magnitude initialization fixed. In the sign-frozen control, $W = \text{sign}(W_{\text{pre}}) \cdot \text{softplus}(V)$ reconstructs the pretrained mixed-sign pattern and preserves it throughout training via a frozen sign buffer, so the resulting FFN is not monotone. In the init-disruption control, $W = V$ is unconstrained but initialized at the same $|W_{\text{pre}}| + \epsilon$ starting point as the monotone arm, isolating recovery training from a sign-disrupted initialization without any nonnegativity constraint. If robustness vanishes under the sign-frozen control, nonnegativity is the causal factor; if it persists, the reparameterization or initialization alone is doing the work. Both arms are screened on seed 42 through the same evaluation and attack stages as the main monotone run, and extended to additional seeds if they move the reported metrics.

4.5 Decoder-Only Foundation Model Track

The T5 experiments above concern an encoder-decoder architecture. To determine whether the monotonicity–robustness relationship generalizes to the decoder-only architectures that dominate contemporary foundation models, we conduct a second experimental track using Pythia-1.4B Biderman et al. (2023), a 24-layer autoregressive language model from the GPT-NeoX family. Models are fine-tuned and evaluated on the Pile Gao et al. (2020) (uncopyrighted subset), and clean performance is measured by perplexity—the exponentiated mean negative log-likelihood per token—on a held-out sample, the standard intrinsic quality metric for autoregressive language models.

Because decoder-only blocks lack the encoder-decoder bottleneck of T5, it is not obvious which sublayers must be constrained for monotonicity to bind. Each GPT-NeoX block contains an attention sublayer with an output projection and a feed-forward (MLP) sublayer with an input projection ($W_{\text{in}} \in \mathbb{R}^{4d \times d}$) and an output projection ($W_{\text{out}} \in \mathbb{R}^{d \times 4d}$). We therefore compare three constraint scopes, each applying the softplus reparameterization of Section 3 directly to the designated weight matrices in all 24 layers (enforcing elementwise nonnegativity, i.e., the coordinatewise order):

- MLP-in: constrain only W_{in}, the direct analogue of the T5 scheme;
- MLP-both: constrain both W_{in} and W_{out};
- MLP-in + Attn-out: constrain W_{in} and the attention output projection.

After imposing constraints, each model undergoes recovery training—fine-tuning on the Pile to re-establish effective representations under the constrained parameterization—with the unconstrained baseline fine-tuned under an identical protocol. Constraint scopes are first screened under a reduced training budget on a single seed; the selected scope is then

validated under the full recovery budget (5 epochs baseline, 10 epochs monotonic) across three seeds (42, 1337, 2024). Robustness is assessed with the same attack implementations as the T5 track: HotFlip with $n = 200$ examples and up to 10 token flips, and UATs optimized on 80 texts and evaluated on 120 disjoint texts, with attack effect measured as the relative increase in negative log-likelihood (NLL) under the learned trigger. Learned UAT triggers are additionally replayed across models to form a 2×2 NLL-increase transfer matrix, and HotFlip substitutions are transferred with the same gradient-free controls as in Section 4.2. The same ordered-pair probe protocol of Section 4.3 is applied to decoder residual streams (last-token pooling as the primary readout, mean pooling as a robustness check) so that end-to-end order preservation is measured on both architectures.

4.6 Scale-Up Protocol

The main-text tables report the T5-small and Pythia-1.4B arms that are already complete. To test whether the quality–robustness pattern persists at larger capacity, the same pipelines accept a model-name override with size-tier presets (batch size, gradient accumulation, bfloat16, and gradient checkpointing). The T5 track is repeated on t5-base (220M) and t5-large (770M) for three seeds with full CNN/DailyMail, XSUM, and SAMSum test sets. The decoder-only track is repeated on Pythia-2.8B for three seeds (MLP-both) and screened on Pythia-6.9B for one seed before a three-seed extension.

A second decoder-only route uses better-known Llama 3 models: Llama-3.2-1B and Llama-3.2-3B for three seeds, with a one-seed screen on Llama-3.1-8B Grattafiori et al. (2024). These models replace the GPT-NeoX MLP with a SwiGLU feed-forward sublayer,

$$N(x) = W_{\text{down}}\big(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x\big).$$

SiLU is not non-decreasing—its derivative is negative on $(-\infty, x^\star)$ for $x^\star \approx -1.28$—and the gate multiplies two input-dependent branches, so Proposition 3.4 does not apply as written. We therefore treat the Llama route as an empirical stress test of the nonnegativity constraint beyond the theory's assumptions, with the Stage 11 order-preservation measurement as the primary test of whether coordinatewise order still accumulates through depth. Two constraint variants are defined: gated-updown constrains W_{up} and W_{down} and leaves W_{gate} free, analogous to the partial scopes used on Pythia; gated-all constrains all three projections. Gated-updown is the default. The same gated-FFN code path also accepts Qwen2.5-1.5B and Qwen2.5-7B as a license-free fallback. These runs use the expanded CURC partitions: NVIDIA H200 (141GB) for the Pythia and Llama arms and NVIDIA RTX Pro 6000 (96GB) for the larger T5 arms, with A100 nodes retained for T5-small and Pythia-1.4B. Results from the scale-up arms replace the modest-scale limitation as they land; they are not substituted into the existing tables until the corresponding JSON artifacts exist.

5 Mechanisms of Gradient Attenuation in Monotone Feed-Forward Networks

We provide a mechanistic explanation for the empirical robustness gains observed in monotone Transformer language models. Rather than claiming global robustness guarantees, we analyze how monotonicity constraints alter the local gradient structure of feed-forward sublayers in a way that can reduce the effectiveness of gradient-based adversarial attacks such as HotFlip.

Our analysis is motivated by classical results in the theory of monotone dynamical systems, which show that, under mild boundedness assumptions, trajectories of strongly monotone flows converge to equilibrium points almost surely; in particular, derivatives along these trajectories tend to zero as the system approaches an invariant set Hirsch (1985). While a feed-forward network is not a time-indexed dynamical system in this sense, this perspective suggests that monotonicity can limit the set of active directions along which outputs and gradients vary. We leverage this intuition to inform a local analysis of gradient behavior under monotone perturbations, focusing on how saturation and non-negative weights interact to diminish exploitable gradient directions.

Consider a monotone FFN update as in Definition 3.5,

$$F(h) := h + N(h), \qquad N(h) = W_2\,\sigma(W_1 h + b_1) + b_2,$$

with $W_1, W_2 \geq 0$ elementwise and σ elementwise non-decreasing. Unlike a subspace-restricted construction, here F acts on the full hidden state $h \in \mathbb{R}^d$, so we analyze the Jacobian of F directly with respect to hidden coordinates, without any auxiliary projection.

Lemma 5.1 (Non-negativity of the FFN Jacobian). If $N(h) = W_2\sigma(W_1 h + b_1) + b_2$ with $W_1, W_2 \geq 0$ elementwise and σ elementwise non-decreasing, then the Jacobian

$$J_F(h) = \nabla_h F(h) = I + J_N(h)$$

has non-negative entries for all $h \in \mathbb{R}^d$.

This follows from the factorization of the FFN Jacobian:

$$J_N(h) = W_2\,\mathrm{diag}(\sigma'(W_1 h + b_1))\,W_1 \geq 0 \quad \text{(elementwise)},$$

since $\sigma'(\cdot) \geq 0$ wherever it exists (non-decreasing σ), hence $J_F(h) = I + J_N(h) \geq 0$ elementwise. Unlike a subspace-projected construction, this holds unconditionally on the full d-dimensional hidden space—no auxiliary projection matrix is involved. Intuitively, increasing any hidden coordinate cannot decrease any output coordinate of F; sign cancellations internal to the FFN sublayer are ruled out entirely.

Let $\mathcal{L} : \mathbb{R}^d \to \mathbb{R}$ be any differentiable scalar objective defined on the sublayer's output (e.g., the loss induced by the rest of the network through $h' = F(h)$). By the chain rule,

$$\nabla_h \mathcal{L}(F(h)) = J_F(h)^\top \nabla_{h'} \mathcal{L}(h')\big|_{h'=F(h)}.$$

Thus the backpropagated gradient is filtered by $J_F(h)^\top$, whose entries are nonnegative under Lemma 5.1.

5.1 Saturation-Induced Gradient Attenuation

We now examine how saturation in monotone networks affects gradient magnitude. Lemma 5.1 holds for any elementwise non-decreasing σ, including the ReLU activation used in our T5-small FFN sublayers. The saturation mechanism analyzed below is a complementary, activation-dependent effect that further attenuates gradients when σ has a bounded derivative; we state it for a general saturating activation, since g-style monotone MLPs in other settings (e.g., sigmoid- or softplus-gated units) exhibit it directly, and remark below on the ReLU case relevant to our experiments.

Assumption 5.2 (Saturating Activations). The activation σ has bounded derivative and admits saturated regimes, i.e., $\sigma'(z)$ becomes arbitrarily small as $z \to +\infty$.

Lemma 5.3 (Gradient Attenuation under Saturation). Let $F(h) = h + N(h)$ where N is an FFN with elementwise nonnegative weights and activation σ satisfying Assumption 5.2. Suppose that along a sequence $\{h_k\}$, a non-empty subset of pre-activations in N diverges to $+\infty$, so that the corresponding entries of σ' converge to 0. Then the contribution of those saturated units to $J_N(h_k)$ vanishes, and hence their contribution to $\nabla_h \mathcal{L}(F(h_k)) = J_F(h_k)^\top \nabla_{h'} \mathcal{L}$ vanishes asymptotically.

Proof is deferred to Appendix A.1.2.

This result shows that monotonicity, when combined with activation saturation, can locally attenuate gradient magnitude without eliminating all gradient signal.

5.2 Persistence of Saturation Effects

The following observation captures a one-sided stability property of saturated units under monotone perturbations.

Lemma 5.4 (Persistence of Saturated Units). Let $h \leq h'$ and consider any first-layer pre-activation $z(h) = (W_1 h + b_1)_j$ within N, with $W_1 \geq 0$ elementwise. If unit j is saturated at h', i.e., $\sigma'(z(h')) = 0$, then for any monotone perturbation $\delta \geq 0$, the unit remains saturated at $h' + \delta$.

Proof is deferred to Appendix A.1.3.

Importantly, this lemma applies to individual hidden units rather than to the entire input gradient, and does not preclude other units from becoming active.

Remark on ReLU. Our T5-small experiments use ReLU FFN activations, for which $\sigma'(z) = 0$ exactly (not merely asymptotically) for all $z < 0$, but $\sigma'(z) = 1$ for all $z > 0$: ReLU saturates on the inactive (negative) side and never saturates on the active (positive) side. Assumption 5.2 and Lemmas 5.3–5.4 as stated describe saturation on the active side, which characterizes bounded activations such as sigmoid or tanh rather than ReLU. For ReLU, the non-negativity result of Lemma 5.1 still holds unconditionally, so the Jacobian-filtering mechanism applies directly to our models; a monotone increase in h can, however, move a previously inactive ReLU unit into its active region rather than deepen an existing saturation, so the persistence property of Lemma 5.4 does not transfer to ReLU without modification. We therefore present the saturation-based mechanism as an additional attenuation effect relevant to monotone networks built from bounded activations, and rely on Lemma 5.1 alone as the mechanism directly applicable to the ReLU-based models studied here; characterizing an analogous persistence property for piecewise-linear activations is left to future work.

5.3 Implications for Gradient-Based Attacks

Gradient-based adversarial attacks such as HotFlip rely on persistent, high-magnitude gradients with respect to token embeddings to identify impactful discrete perturbations. The preceding analysis suggests that monotonicity constraints reduce the availability of such gradients in two ways: directly, by ruling out sign cancellation in the FFN Jacobian for any non-decreasing activation (Lemma 5.1), and, for networks built from bounded activations, by driving subsets of hidden units into saturated regimes from which they do not recover under monotone increases in internal representations (Lemmas 5.3–5.4).

While monotone feed-forward networks are not dynamical systems in the formal sense, their behavior is reminiscent of classical results in monotone dynamical systems, where cooperative interactions restrict the effective directions along which trajectories evolve. This analogy is intended as intuition rather than a formal guarantee: monotonicity does not prevent all adversarial attacks, nor do gradients vanish globally. Instead, our analysis identifies local attenuation effects that weaken gradient-based optimization strategies, offering a plausible mechanism for the empirical robustness improvements observed in Section 6. Section 6.3 probes this mechanistic account directly by testing whether the robustness gain survives gradient-free and cross-model attack transfer, as a purely gradient-attenuation account predicts it should if the effect is specific to each model's own loss landscape.

6 Results

We evaluate the impact of monotonicity constraints on both task performance and adversarial robustness. We first analyze optimization behavior and summarization quality on the encoder-decoder (T5) track to assess potential performance degradation, and then evaluate robustness under targeted adversarial attacks. We conclude with the decoder-only foundation model track (Section 6.4), which tests whether the robustness benefit transfers across architectures. Together, these experiments assess whether monotonicity serves as a practical inductive bias for improving robustness without sacrificing model utility.

6.1 Training Dynamics and Summarization Quality

We begin by analyzing the effect of monotonicity constraints on optimization behavior and summarization quality. Table 1 summarizes the training dynamics of the baseline and monotonic models from our experiments. The baseline model refers to a T5-small checkpoint fine-tuned under identical settings as the monotonic model but without monotonicity constraints, while the standard model denotes the pretrained T5-small without additional fine-tuning. The monotonic model starts with a substantially higher initial loss (4.97 vs. 2.94), reflecting the magnitude-preserving but sign-flipping softplus initialization described

Table 1: Training dynamics comparison (seed = 42).

Model	Initial Loss	Final Loss	Δ Loss	Epochs
Baseline	2.94	2.21	−0.73	6
Monotonic	4.97	2.54	−2.43	7

Table 2: Summarization quality on CNN/DailyMail (n = 200, seed = 42). Values are means with 95% bootstrap confidence intervals.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Standard	32.6 [30.9, 34.2]	11.9 [10.5, 13.4]	26.6 [25.0, 28.1]
Baseline	30.9 [29.4, 32.4]	11.5 [10.1, 12.8]	25.0 [23.6, 26.4]
Monotonic	29.6 [28.0, 31.1]	10.6 [9.2, 11.9]	24.2 [22.7, 25.6]

in Section 3: since roughly half of each pretrained FFN weight matrix is negative, enforcing $W \geq 0$ while preserving $|W|$ changes the function computed by every FFN sublayer substantially. This initialization disrupts the original weight geometry, requiring the model to re-establish effective internal representations during fine-tuning.

Despite this unfavorable initialization, the monotonic model converges reliably during training. Validation loss decreases from 2.92 at epoch 1 to 2.54 at epoch 7, indicating stable optimization under the imposed constraints. The model exhibits rapid initial recovery, reducing training loss by 1.92 points in the first two epochs (from 4.97 to 3.05), followed by slower asymptotic convergence with diminishing returns in later epochs. A persistent optimization gap of 0.32 points remains at convergence (2.54 vs. 2.21 for baseline), consistent with the reduced expressivity of the constrained parameter space. This gap reflects a deliberate and quantifiable trade-off: the baseline model achieves slightly lower validation loss through unrestricted FFN weights, while the monotonic model maintains competitive performance while enforcing structural regularity that, as shown below, translates to improved adversarial robustness.

Table 2 reports summarization performance on CNN/DailyMail (n=200, seed 42). The monotonic model achieves a ROUGE-L score of 24.2 [22.7, 25.6], compared to 25.0 [23.6, 26.4] for the baseline, corresponding to a relative decrease of 3.2 percent. ROUGE-1 exhibits a similar trend (29.6 [28.0, 31.1] vs 30.9 [29.4, 32.4], a 4.2 percent decrease). While these results indicate a modest reduction in task performance, the monotonic model remains competitive, with confidence intervals showing substantial overlap across all metrics.

To assess generalization, we additionally evaluate on XSUM (n=200, seed 42). Table 3 shows consistent results: the monotonic model achieves ROUGE-L of 12.9 [12.3, 13.6] compared to 13.3 [12.6, 13.9] for baseline, a similar 3.0 percent relative gap. The consistency of the performance trade-off across distinct summarization datasets (news articles in CNN/DM, single-sentence summaries in XSUM) suggests that the cost of monotonicity constraints is stable and predictable across different data distributions.

The training dynamics exhibit a characteristic pattern of monotone-constrained optimization. During the first two epochs, the monotonic model rapidly recovers much of the performance gap introduced by constrained initialization, reducing training loss by 1.92 points (from 4.97 to 3.05). Subsequent epochs yield diminishing returns, with only 0.36 points of additional improvement from epochs 3 through 7. This behavior highlights the importance of the extended warmup phase (15 percent vs 10 percent for baseline), which stabilizes gradient estimates and enables effective learning despite the constrained parameter space.

Analysis of generation behavior reveals that both fine-tuned models produce longer outputs than the pretrained T5 model (mean lengths 74 to 76 tokens vs 57 tokens), reflecting adaptation to the distributional properties of the summarization training data. The monotonic model exhibits slightly higher length variance (std 11.3 tokens) compared to the standard

Table 3: Summarization quality on XSUM (n = 200, seed = 42). Values are means with 95% bootstrap confidence intervals.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Standard	19.9 [18.8, 21.1]	2.9 [2.3, 3.4]	13.3 [12.6, 14.1]
Baseline	20.0 [19.0, 21.0]	3.2 [2.7, 3.7]	13.3 [12.6, 13.9]
Monotonic	18.9 [18.0, 19.7]	2.8 [2.3, 3.2]	12.9 [12.3, 13.6]

Table 4: HotFlip attack results on CNN/DailyMail (n = 100, seed = 42).

Model	Avg. Deg.	Success Rate	ΔLoss
Standard	21.3%	69%	+0.42
Baseline	16.2%	63%	+0.35
Monotonic	5.2%	19%	+0.14

model (std 12.0 tokens), though this difference is minimal. Importantly, the brevity penalty for the monotonic model (1.00) indicates that output length is well-calibrated to reference summary length, suggesting the constraints do not introduce systematic length bias.

6.2 Adversarial Robustness

HotFlip Attacks. We evaluate gradient-based robustness using HotFlip attacks, which identify vulnerable input positions via gradients with respect to token embeddings and perform targeted token substitutions to maximize loss. For each example, the attack computes gradients of the cross-entropy loss, selects the five positions with largest gradient magnitudes, and replaces tokens at those positions with vocabulary items that maximize the dot product between the gradient and the replacement embedding.

Robustness is quantified using three complementary metrics: average degradation, the mean relative loss increase under attack; attack success rate, the fraction of examples for which degradation exceeds 10%; and mean loss increase, the average absolute loss increase.

Table 4 reports results on 100 test examples from seed 42. To verify robustness consistency, we repeated the HotFlip evaluation using the same checkpoints with two additional random seeds (1337, 2024) for attack sampling variability. Results were identical across all three seeds, confirming that the observed robustness improvements are stable. The monotonic model demonstrates improved robustness relative to both baselines. Under HotFlip perturbations, the monotonic model incurs an average degradation of 5.2 percent, compared to 16.2 percent for the fine-tuned baseline and 21.3 percent for the standard pretrained model. Attack success rates follow a similar pattern, with only 19 percent of attacks succeeding against the monotonic model, compared to 63 percent for the baseline and 69 percent for the standard model. This corresponds to a 69.8 percent relative reduction in attack success rate (63 percent to 19 percent) and a 67.9 percent reduction in average degradation (16.2 percent to 5.2 percent). Paired t-tests confirm that the difference between the baseline and monotonic models is highly significant (t=6.17, $p = 3.8\times10^{-9}$), with an even larger effect relative to the standard model (t=7.45, p=2.8×10^{-12}).

Overall, enforcing monotonicity in the FFN sublayers yields a 69.8 percent relative reduction in attack success rate (from 63 percent to 19 percent) and a 67.8 percent reduction in average degradation (from 16.1 percent to 5.2 percent), without requiring adversarial training, modified training objectives, or architectural changes beyond the FFN constraints. The effect size is large (Cohen's $d > 0.8$) and the statistical significance is robust ($p < 10^{-9}$). These results demonstrate that structural constraints alone—applied selectively to feed-forward sublayers—can substantially improve robustness to gradient-based adversarial manipulation,

Table 5: UAT trigger transfer matrix (seed = 42; $n = 100$). Entry (i, j) is the ROUGE-L delta (percentage points, attacked minus clean) when the trigger learned on model i is evaluated against model j. Diagonal entries (bold) are same-model attacks.

Trigger source \ Target	Standard	Baseline	Monotonic
Standard	**−0.40**	−0.45	−0.62
Baseline	−0.36	**+0.06**	+0.26
Monotonic	+0.42	+0.15	**−0.53**

providing evidence for monotonicity as a practical architectural inductive bias for building more robust language models.

Universal Adversarial Trigger Attacks. We evaluate robustness to universal adversarial triggers (UATs), input-agnostic token sequences optimized to maximize model loss when prepended to any input. Triggers are learned via coordinate ascent with 3 restarts and 50 iterations over 80 training examples, then evaluated on 120 disjoint test examples. Robustness is measured via NLL increase under trigger prepending. UAT attacks prove minimally effective across all models, with NLL increases below 1 percent. The monotonic model exhibits the smallest degradation (0.73 percent NLL increase), compared to 0.89 percent for the baseline and 0.97 percent for standard T5. While this ordering is consistent with improved robustness, the effect sizes are too small to draw strong conclusions. The weak effectiveness of universal triggers, in contrast to the impact of input-specific HotFlip attacks, suggests that input-agnostic perturbations are limited for attacking sequence-to-sequence models, and that monotonicity's robustness benefits are most pronounced against adaptive, gradient-based attacks.

6.3 Trigger Transferability

A HotFlip-style success-rate gap could in principle reflect a genuine difference in each model's function, or an artifact of the fact that each model's attack is optimized against its own loss landscape. UATs are well suited to a first check of this distinction: each trigger is learned independently per model, so evaluating a trigger learned on model A against model B tests whether the trigger transfers, i.e., whether it captures a vulnerability specific to A's loss landscape or a shared weakness that is not model-specific. Table 5 reports the full 3×3 transfer matrix (ROUGE-L delta on a held-out 100-example subset) using each model's own learned trigger (rows) evaluated against every model (columns). The corresponding matrices stored for seeds 1337 and 2024 match these entries to the reported precision.

Every entry lies within ± 0.62 percentage points of ROUGE-L, an order of magnitude smaller than the HotFlip degradations reported above, and the sign of each entry is not consistent within a row or column: the monotonic model's own trigger is mildly more damaging to the standard and baseline models than to the monotonic model itself, while the standard model's trigger is comparably (in fact slightly more) damaging to the monotonic model than to the standard model. There is no row or column whose entries are systematically larger or smaller than the others. Under UATs, in other words, no model is a systematically easier or harder target than any other, whether attacked by its own trigger or by another model's, which is the pattern expected if the near-zero UAT effect sizes documented above reflect UATs being a weak attack against sequence-to-sequence summarization models in general rather than a per-model vulnerability that happens to differ between the monotonic and non-monotonic models. Because UAT trigger search here is itself a gradient-free coordinate ascent over discrete tokens rather than a first-order attack, this transfer check complements, rather than substitutes for, the HotFlip substitution-transfer and gradient-free controls of Section 4.2, which probe the same distinction on the attack family that produces the larger and more differential effect sizes.

Table 6: Constraint-scope screening on Pythia-1.4B (seed = 42). PPL denotes held-out Pile perplexity; SR denotes HotFlip attack success rate ($n = 200$, 10 flips). MLP-in was screened under the full training budget, the other scopes under a reduced budget.

Constraint scope	PPL base	PPL mono	SR base → mono	SR drop
MLP-in	6.67	24.32	71.5% → 83.5%	−12.0 pp
MLP-both	7.06	47.99	75.0% → 44.5%	+30.5 pp
MLP-in + Attn-out	7.06	88.17	75.0% → 35.5%	+39.5 pp

Table 7: Pythia-1.4B results with the MLP-both constraint scope under the full recovery budget. PPL denotes held-out Pile perplexity; SR denotes HotFlip attack success rate ($n = 200$, 10 flips); UAT NLL↑ denotes the relative NLL increase under the learned universal trigger (base → monotone).

Seed	PPL base	PPL mono	SR base → mono	UAT NLL↑
42	6.72	36.01	69.0% → 43.5%	+15.5% → +8.8%
1337	6.72	42.00	69.0% → 39.0%	+16.4% → +8.1%
2024	6.72	35.03	69.0% → 45.0%	+16.5% → +15.0%
Mean	6.72	37.68	69.0% → 42.5%	+16.1% → +10.6%

6.4 Generalization to Decoder-Only Foundation Models

We next ask whether the robustness benefit of monotone feed-forward constraints is specific to the encoder-decoder setting or extends to decoder-only foundation models, using the Pythia-1.4B track described in Section 4.5.

Constraint scope determines whether robustness transfers. Table 6 reports the constraint-scope screening. The direct analogue of the T5 scheme—constraining only the MLP input projection (MLP-in)—fails on the decoder-only architecture: perplexity increases by a factor of 3.65, yet the HotFlip attack success rate rises from 71.5% to 83.5%. The constraint degrades clean performance without impeding the attack, indicating that adversarial gradient signal is routed around the constrained matrix through the unconstrained MLP output projection and attention pathways. Extending the constraint scope closes these leak paths: constraining both MLP projections (MLP-both) reduces the success rate by 30.5 percentage points, and additionally constraining the attention output projection (MLP-in + Attn-out) yields a 39.5-point reduction. The latter, however, roughly doubles the perplexity penalty (12.49× versus 6.80×) for a further 9-point reduction in success rate, a diminishing return consistent with attention serving as a secondary leak path. Balancing robustness against clean performance, we select MLP-both for full-budget validation.

Multi-seed validation. Table 7 reports the MLP-both scope under the full recovery-training budget across three seeds. The robustness improvement is stable: the HotFlip success rate falls from 69.0% to between 39.0% and 45.0% (mean 42.5%), an absolute reduction of 26.5 percentage points and a relative reduction of 38.4%. Under UAT attacks—which, unlike on T5, are substantially damaging to the decoder-only baseline, raising NLL by approximately 16%—the monotone models are consistently less affected, with a mean NLL increase of 10.6% versus 16.1% for the baseline. The monotone models thus improve robustness against both adaptive gradient-based and input-agnostic attacks in this setting.

The quality–robustness trade-off is architecture-dependent. The robustness gains on Pythia-1.4B come at a markedly higher cost in clean performance than on T5. Held-out perplexity increases by a mean factor of 5.61 (range 5.21–6.25), and full-budget recovery training does not close this gap, whereas the T5 monotone model recovers to within a few percent of its baseline. We attribute this asymmetry to architectural structure: in T5, the encoder-decoder bottleneck and cross-attention alignment restrict the routes by which token-level

perturbations can propagate without traversing a constrained sublayer, so constraining the feed-forward input projections suffices and leaves most of the network free to recover expressivity. In a decoder-only model, autoregressive self-attention connects every position to all preceding positions without passing through any constrained layer, providing a bypass for adversarial signal; robustness therefore requires constraining a larger fraction of each block's transformation, which in turn imposes a steeper expressivity penalty. The monotonicity–robustness mechanism itself transfers across architectures, but the constraint scope required to realize it—and the price paid in clean performance—is architecture-conditional.

7 Discussion

This work provides empirical evidence and theoretical analysis supporting monotonicity as a structural bias for improving robustness in modern language models. By selectively enforcing monotonicity in feed-forward sublayers, we observe consistent reductions in vulnerability to adversarial and jailbreak-style attacks—while largely preserving task performance in the encoder-decoder setting. These gains arise without additional data, modified training objectives, or heuristic defenses, highlighting architectural structure as a direct lever for shaping model behavior under perturbation.

Our theoretical analysis offers insight into why these gains arise, even in the absence of formal end-to-end robustness guarantees. Viewing a language model as a high-dimensional system whose internal semantic state evolves through successive nonlinear transformations, monotonicity constrains how perturbations can propagate across layers. In particular, directional consistency ensures that strengthening information or constraints in internal representations cannot induce regressions downstream. This restriction narrows the range of admissible behaviors and simplifies reasoning about worst-case effects, which must occur at semantic extremes rather than intermediate variations.

Empirically, the robustness gains are closely tied to the role of feed-forward sublayers. While attention mechanisms perform semantic aggregation and contextual reasoning, FFNs dominate nonlinear transformation and amplification. Enforcing monotonicity at this stage constrains how semantic information is refined after context has been established, without interfering with attention or global information flow. This selective enforcement alters the local gradient structure of the network, reducing the stability and magnitude of exploitable gradients used by attacks such as HotFlip.

These observations align with classical insights from monotone systems theory, where structural monotonicity limits perturbation propagation and enables tractable analysis of global behavior in high-dimensional systems. Although Transformer models are not dynamical systems in a formal sense, the analogy provides intuition and motivates our analysis of monotonic components within the network.

The decoder-only experiments sharpen this picture by showing that the leak-path argument is architectural rather than incidental. Where the encoder-decoder structure of T5 funnels perturbations through constrained sublayers, the residual stream of a decoder-only model offers unconstrained routes around any single constrained matrix; constraining only the feed-forward input projection accordingly fails on Pythia-1.4B, while constraining both feed-forward projections restores the robustness benefit. Monotonicity should therefore be understood not as a property of an isolated sublayer but of the set of paths through which adversarial signal can propagate—an observation that directly informs where constraints must be placed in a given architecture.

In the encoder-decoder setting, the trade-off introduced by monotonicity is modest and predictable: constraining FFN weights slightly reduces expressivity, and the resulting decrease in summarization quality is small relative to the robustness improvements. In the decoder-only setting, the broader constraint coverage required for robustness carries a substantially larger perplexity cost that full-budget recovery training does not eliminate, and reducing this cost—through better initialization, specialized training schedules, or partial-layer application—is a concrete open problem. More broadly, monotonicity differs from conventional regularization in that it constrains how representations may change, rather than

simply penalizing their magnitude, promoting predictability rather than uniform smoothness.

The results reported in the main tables are from T5-small and Pythia-1.4B. A scale-up protocol (Section 4.6) repeats the same evaluation on T5-base, T5-large, Pythia-2.8B, a Pythia-6.9B screen, and a Llama 3 gated-FFN route (1B/3B with an 8B screen); those numbers are inserted only after the corresponding runs finish. The Llama arms test whether robustness and order-preservation effects persist when the feed-forward activation is SwiGLU rather than a monotone σ, a setting outside Proposition 3.4. The study does not provide formal robustness certificates. The HotFlip substitution-transfer and gradient-free controls of Section 4.2, the end-to-end order-preservation measurement of Section 4.3, and the sign-frozen and init-disruption ablations of Section 4.4 isolate whether the reported robustness gap is a property of each model's function, whether coordinatewise FFN monotonicity induces semantic order through depth, and whether nonnegativity (rather than reparameterization or initialization disruption) is the causal factor. Extending the theoretical analysis toward stronger guarantees, scaling the approach to larger models, and narrowing the quality gap on decoder-only architectures are important directions for future work. Nonetheless, the combination of analytical insight and consistent empirical gains across two architecture families suggests that monotonicity is a principled and scalable design choice for improving the robustness and predictability of large language models.

8 Conclusion

We investigated monotonicity as a structural inductive bias for improving robustness in Transformer language models. By enforcing non-negativity constraints on feed-forward sublayers, we observe substantial reductions in vulnerability to gradient-based adversarial attacks with only modest impact on summarization quality in the encoder-decoder setting. Extending the study to a decoder-only foundation model shows that the benefit transfers, but only when the constraint scope is matched to the architecture: decoder-only models require constraining both feed-forward projections to close adversarial leak paths, and incur a larger cost in clean perplexity. These results suggest that simple architectural constraints can meaningfully improve robustness without altering attention mechanisms or relying on adversarial training, provided constraint placement accounts for how signal propagates through the architecture.

An important direction for future work is to develop theoretical guarantees that characterize when and why monotonicity yields improved robustness in high-dimensional language models. Reducing the perplexity cost of monotone constraints on decoder-only architectures, and scaling this approach to larger model variants and broader tasks, are further promising avenues that may help clarify the role of structural constraints in building more reliable language models.

Ethics statement

This work explores architectural constraints that improve the robustness of language models to adversarial manipulation. By studying monotonicity as a structural inductive bias, our approach contributes to the development of language models whose behavior is more predictable and analyzable, which is particularly relevant for safety-critical and decision-support applications. All experiments use publicly available pretrained checkpoints (T5, Pythia, Llama 3) and public benchmark datasets (CNN/DailyMail, XSUM, SAMSum, the Pile); the work does not involve human subjects or private data. The adversarial attacks studied (UAT, HotFlip) are standard robustness-evaluation techniques applied only to models under the authors' control, not a mechanism for attacking deployed third-party systems.

Reproducibility statement

Full training, evaluation, and attack configurations (learning rate, batch size, warmup schedule, seeds, decoding parameters) are given in Section ??. All reported confidence intervals

use bootstrap resampling (1,000 resamples) and significance is assessed via paired t-tests with Bonferroni correction, as implemented in hpc_version/utils/stats_utils.py. Every reported run is tagged with its exact random seed and timestamp metadata.

AUTHOR: add the anonymized code/checkpoint release plan here (e.g., an anonymous repository link) before submission.

AI use statement

AUTHOR: review and finalize this statement before submission – confirm the scope below matches actual usage.

Generative AI tools (Claude, via Claude Code) were used to assist with software engineering tasks in this project, including developing and debugging the SLURM job-orchestration pipeline, experiment-tracking and results-aggregation scripts, and the automated test suite, and for editorial assistance in drafting and revising manuscript text. AI tools were not used to generate the theoretical claims, proofs, or experimental results reported in this paper; all reported numbers are produced by the training/evaluation/attack pipeline in hpc_version/ and foundation_llm_experiments/ and were verified by the authors against the underlying result JSON files before inclusion. All AI-assisted code and text were reviewed by the authors, who take full responsibility for the final content of this work, including any text, claims, or artifacts produced with the aid of generative AI.

References

Brandon Amos and J. Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In International Conference on Machine Learning (ICML), 2017.

Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel Ford, et al. Many-shot jailbreaking. Advances in Neural Information Processing Systems, 37:129696–129742, 2024.

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In Proceedings of the 40th International Conference on Machine Learning, volume 202, pp. 2397–2430. PMLR, 2023.

Bochuan Cao, Yuanpu Cao, Lu Lin, and Jinghui Chen. Defending against alignment-breaking attacks via robustly aligned llm. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 10542–10560, 2024.

Hani Daniels and Marina Velikova. Monotone and partially monotone neural networks. IEEE Transactions on Neural Networks, 21(6):906–917, 2010.

Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. Multilingual jailbreak challenges in large language models. 2023. ArXiv, abs/2310.06474, 2024.

Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. HotFlip: White-box adversarial examples for text classification. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 31–36. Association for Computational Linguistics, 2018.

Hamid Reza Feyzmahdavian, Bart Besselink, and Mikael Johansson. Stability analysis of monotone systems via max-separable lyapunov functions. IEEE Transactions on Automatic Control, 63(3):643–656, 2017.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.

Maya Gupta, Andrew Cotter, Jan Pfeifer, and Konstantin Voevodski. Monotonic calibrated interpolated look-up tables. In Advances in Neural Information Processing Systems (NeurIPS), 2016.

Morris W Hirsch. Systems of differential equations that are competitive or cooperative ii: Convergence almost everywhere. SIAM Journal on Mathematical Analysis, 16(3):423–439, 1985.

Nikolaus HR Howe, Ian R McKenzie, Oskar John Hollinsworth, Michał Zajac, Tom Tseng, Aaron David Tucker, Pierre-Luc Bacon, and Adam Gleave. Effects of scale on language model robustness. 2024.

Hiroshi Ito, Björn S Rüffer, and Anders Rantzer. Max-and sum-separable lyapunov functions for monotone systems and their level sets. In 53rd IEEE Conference on Decision and Control, pp. 2371–2377. IEEE, 2014.

Stasys Jukna et al. Boolean function complexity: advances and frontiers, volume 27. Springer, 2012.

David Khachaturov and Robert Mullins. Adversarial suffix filtering: a defense pipeline for llms. arXiv preprint arXiv:2505.09602, 2025.

Miles Q Li and Benjamin CM Fung. Security concerns for large language models: A survey. Journal of Information Security and Applications, 95:104284, 2025.

Anthony N Michel, Ling Hou, and Derong Liu. Stability of dynamical systems: On the role of monotonic and non-monotonic Lyapunov functions. Springer, 2015.

Laker Newhouse, R Preston Hess, Franz Cesista, Andrii Zahorodnii, Jeremy Bernstein, and Phillip Isola. Training transformers with enforced lipschitz constants. arXiv preprint arXiv:2507.13338, 2025.

An-Phi Nguyen, Dana Lea Moreno, Nicolas Le-Bel, and María Rodríguez Martínez. Mononet: enhancing interpretability in neural networks via monotonic features. Bioinformatics advances, 3(1):vbad016, 2023.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of machine learning research, 21(140):1–67, 2020.

Anders Rantzer and Bo Bernhardsson. Control of convex-monotone systems. In 53rd IEEE Conference on Decision and Control, pp. 2378–2383. IEEE, 2014.

Alexander Robey, Zifan Wang, and J. Zico Kolter. Smoothllm: Defending large language models against jailbreak attacks. arXiv preprint arXiv:2310.03684, 2023.

Davor Runje and Sharath M Shankaranarayana. Constrained monotonic neural networks. In International Conference on Machine Learning, pp. 29338–29353. PMLR, 2023.

Ahmed Ragab Mahmoud Salah. Jailbreaking llms: An in-depth security analysis of prompt injection vulnerabilities. 2026.

Davide Sartor, Alberto Sinigaglia, and Gian Antonio Susto. Advancing constrained monotonic neural networks: Achieving universal approximation beyond bounded activations. arXiv preprint arXiv:2505.02537, 2025.

Md Faiyaz Abdullah Sayeedi, Maaz Bin Hossain, Md Kamrul Hassan, Sabrina Afrin, Molla Md Sabit, and Md Shohrab Hossain. Jailbreaktracer: Explainable detection of jailbreaking prompts in llms using synthetic data generation. IEEE Access, 2025.

Joseph Sill. Monotonic networks. In Advances in Neural Information Processing Systems (NeurIPS), 1998.

Hal L Smith. Monotone dynamical systems: an introduction to the theory of competitive and cooperative systems. Number 41. American Mathematical Soc., 1995.

Jingtong Su, Julia Kempe, and Karen Ullrich. Mission impossible: A statistical perspective on jailbreaking llms. Advances in Neural Information Processing Systems, 37:38267–38306, 2024.

Eric Wallace, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. Universal adversarial triggers for attacking and analyzing NLP. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 2153–2162. Association for Computational Linguistics, 2019.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, and Samuel R. Bowman. Adversarial glue: A multi-task benchmark for robustness evaluation of language models. In Advances in Neural Information Processing Systems (NeurIPS), 2021.

Haohua Wang, Jingge Wang, Zijie Zhao, Yang Tan, Yanru Wu, Hanbing Liu, Jingyun Yang, Enming Zhang, Xiangyu Chen, Zhengze Rong, et al. Understanding knowledge transferability for transfer learning: A survey. arXiv preprint arXiv:2507.03175, 2025a.

Yiwei Wang, Muhao Chen, Nanyun Peng, and Kai-Wei Chang. Vulnerability of large language models to output prefix jailbreaks: Impact of positions on safety. In Findings of the Association for Computational Linguistics: NAACL 2025, pp. 3939–3952, 2025b.

Maurice Weber, Randall Balestriero, and Richard Baraniuk. Certifying monotonic neural networks. arXiv preprint arXiv:2102.12566, 2021.

Siyu You, Yanzhi Ding, Marco Canini, Jan Pfeifer, and Maya Gupta. Deep lattice networks and partial monotonic functions. In Advances in Neural Information Processing Systems (NeurIPS), 2017.

Yifan Zhu, Yue Xie, Xin Chen, and Qiang Zhang. Autodan: Generating stealthy jailbreak prompts for large language models. In Advances in Neural Information Processing Systems (NeurIPS), 2023.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, and J. Zico Kolter. Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043, 2023.

A Appendices

A.1 Mathematical Proofs

A.1.1 Proof of Lemma 3.3

Proof. For any $x, x' \in \mathbb{R}^n$ such that $x \leq x'$, monotonicity of f implies $f(x) \leq f(x')$. Applying monotonicity of g yields

$$g(f(x)) \leq g(f(x')),$$

which establishes the claim. □

A.1.2 Proof of Lemma 5.1

Proof. Recall $F(h) = h + N(h)$, so

$$J_F(h) = \nabla_h F(h) = I + J_N(h).$$

For $N(h) = W_2\sigma(W_1h + b_1) + b_2$ with $W_1, W_2 \geq 0$ elementwise,

$$J_N(h) = W_2 \operatorname{diag}(\sigma'(W_1h + b_1))W_1.$$

Since σ is elementwise non-decreasing, $\sigma'(z) \geq 0$ wherever it exists, so $\operatorname{diag}(\sigma'(\cdot)) \geq 0$ elementwise. Hence $J_N(h) \geq 0$ elementwise, and therefore $J_F(h) = I + J_N(h) \geq 0$ elementwise. □

A.1.3 Proof of Lemma 5.4

Proof. Let $z_j(h) := (W_1h + b_1)_j$. Since $W_1 \geq 0$ elementwise and $\delta \geq 0$,

$$z_j(h' + \delta) = (W_1(h' + \delta) + b_1)_j \geq (W_1h' + b_1)_j = z_j(h').$$

Thus if the unit is saturated at h' (i.e., $\sigma'(z_j(h')) = 0$ in the saturated regime), it remains saturated at $h' + \delta$. □

A.2 Training and Evaluation Protocol

Both baseline and monotone models are fine-tuned from the same pretrained checkpoint under identical optimization settings. The main-text T5 tables use T5-small. Larger T5 arms (Section 4.6) keep the same optimizer and epoch budget and scale the effective batch size through gradient accumulation. We use the AdamW optimizer with learning rate 5×10^{-5} and weight decay 0.01, a per-device batch size of 4 on T5-small (reduced on larger tiers), and gradient clipping with norm 1.0. To accommodate the constrained parameterization, the monotone model employs an extended warmup phase of 15% of training steps, compared to 10% for the baseline. Both models are trained for 7 epochs and receive the same total training budget.

Training is conducted on the DialogSum, HighlightSum, and arXiv abstract datasets, comprising approximately 150K examples in total. Evaluation is performed on three held-out benchmarks: CNN/DailyMail (11,490 test examples), XSUM (11,334 test examples), and SAMSum (819 test examples). CNN/DailyMail is excluded entirely from training to assess out-of-distribution generalization.

At inference time, decoding is performed using beam search with 4 beams and a length penalty of 1.2, enforcing a minimum generation length of 10 tokens and a maximum length of 80 tokens. We additionally apply no-repeat n-gram blocking with $n = 3$ to discourage degenerate repetitions. All decoding parameters are fixed across models and evaluation sets. ROUGE-1, ROUGE-2, and ROUGE-L scores are computed using the rouge-score library with Porter stemming. We report bootstrap-based 95% confidence intervals using 1,000 resamples and assess statistical significance via paired t-tests with Bonferroni correction for multiple comparisons. Effect sizes are reported using Cohen's d.

A.3 Experimental Setup

The T5 tables in the main text report seed 42 on $n = 200$ evaluation subsets (CNN/DailyMail and XSUM). The validation protocol repeats the pipeline on five seeds (42, 1337, 2024, 8888, and 12345) with the full held-out test sets (CNN/DailyMail 11,490, XSUM 11,334, SAMSum 819), controlling randomness across Python, NumPy, and PyTorch. Cross-seed significance uses paired t-tests on seed-level means with Bonferroni correction across the metric family and paired Cohen's d; within each seed, per-example paired t-tests are computed on the ROUGE vectors of the baseline and monotone models. Completed five-seed and full-test numbers replace the subset tables when those JSON artifacts exist. Experiments run with deterministic algorithms where available, on NVIDIA A100 (40GB and 80GB), H200 (141GB), and RTX Pro 6000 (96GB) GPUs. The A100 environment uses PyTorch with CUDA 11.8; H200 and RTX Pro 6000 nodes use a CUDA 12.8 PyTorch build. Transformers 4.30 or later is used throughout.

A.4 Foundation Model Training Protocol

The decoder-only track fine-tunes Pythia-1.4B Biderman et al. (2023) on the uncopyrighted subset of the Pile Gao et al. (2020), with held-out evaluation texts drawn from a disjoint streaming sample. Constraint-scope screening (Table 6) uses a reduced budget of 30,000 training samples at sequence length 1024 for 3–4 epochs on seed 42; full-budget validation of the selected MLP-both scope (Table 7) trains the baseline for 5 epochs and the monotone model for 10 epochs on each of three seeds (42, 1337, 2024). The extended monotone budget parallels the extended warmup used in the T5 track and compensates for the constrained parameterization's slower recovery. HotFlip evaluation uses 200 held-out examples with up to 10 token flips per example; UAT evaluation uses coordinate ascent with 3 restarts and 50 iterations, optimizing triggers on 80 texts and evaluating on 120 disjoint texts. Attack-time model evaluation is performed in bfloat16 precision. The Llama 3 scale-up route (Section 4.6) uses the same recovery budget, attack protocol, and order-preservation measurement, with the gated-updown constraint as the default scope. Foundation-model experiments run on NVIDIA A100 (80GB) for Pythia-1.4B and on NVIDIA H200 (141GB) for the Pythia-2.8B/6.9B and Llama arms, under the SLURM workload manager, with per-stage checkpointing to permit resumption across scheduler preemptions.